Clustering Algorithms in Machine Learning

Taxonomy, Mathematics, Diagrams & Examples

Machine Learning Course

Department of Computer Science

May 11, 2026

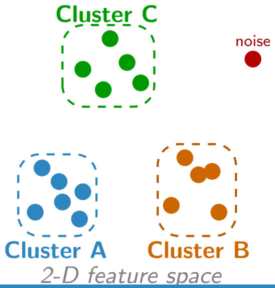
Table of Contents

Table of Contents

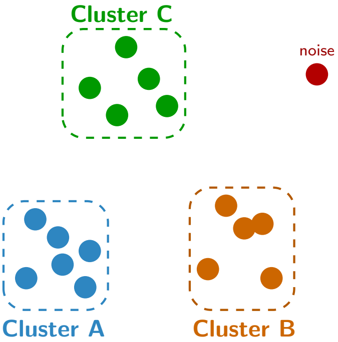
What is Clustering?

Definition

Clustering is an *unsupervised* machine learning task that groups a set of objects so that objects within the same group (**cluster**) are more *similar* to each other than to those in other groups — without using predefined labels.



What is Clustering?



2-D feature space

- **Unsupervised:** no labels required
- **Goal:** maximise intra-cluster similarity and inter-cluster dissimilarity
- **Distance metrics:** Euclidean, cosine, Manhattan, . . .
- **Applications:**
  - Customer / market segmentation
  - Anomaly & fraud detection
  - Document & image grouping
  - Bioinformatics (gene expression)
  - Social-network community detection

Taxonomy of Clustering Algorithms

Clustering Methods

Partitioning
- k-Means
- k-Medoids
- Fuzzy C-Means

Hierarchical
- Agglomerative
- Divisive
- BIRCH

Density-Based
- DBSCAN
- OPTICS
- HDBSCAN

Model-Based
- GMM / EM
- Bayesian GMM
- SOM

Grid-Based
- STING
- CLIQUE
- WaveCluster

Family	Shape	Specify k?	Handles Noise	Complexity
Partitioning	Spherical	Yes	No	$O(nkdi)$
Hierarchical	Arbitrary	Post-hoc	Partial	$O(n^2 \log n)$
Density-Based	Arbitrary	No	Yes	$O(n \log n)$
Model-Based	Elliptical	Yes	Partial	$O(nkdi)$

Table of Contents

k-Means Clustering — Overview

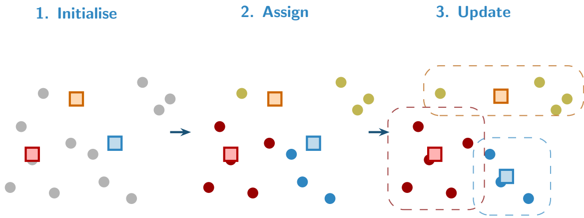
Objective Function

Minimise the **Within-Cluster Sum of Squares** (WCSS):

$$J = \sum_{i=1}^{k} \sum_{\mathbf{x} \in C_i} \left\| \mathbf{x} - \boldsymbol{\mu}_i \right\|^2$$

where $\boldsymbol{\mu}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$ is the *centroid*.

Lloyd's Algorithm — 3-Step Iteration



k-Means Clustering — Overview

Distance Assignment

Point $\mathbf{x}$ is assigned to cluster C_i^*:

$$i^* = \arg\min_{1 \le i \le k} \left\| \mathbf{x} - \boldsymbol{\mu}_i \right\|^2$$

Complexity: $O(n \cdot k \cdot d \cdot t)$ n=points, k=clusters, d=dims, t=iterations.
Convergence: guaranteed (WCSS is non-increasing).

Example: Customer Segmentation

Data: customers described by *age* & *annual spend*.
Result ($k = 3$): Budget | Mid-tier | Premium shoppers.

k-Means — Algorithm & Properties

Input: Data $X = \{x_1, \ldots, x_n\}$, clusters k
Output: Cluster assignments C, centroids μ
Initialise $\mu_1, \ldots, \mu_k$ (random or k-means++);
repeat
  for *each point* x_i **do**
    $C(i) \leftarrow \arg\min_j \|x_i - \mu_j\|^2$;
  end
  for *each cluster* j **do**
    $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$;
  end
until *centroids unchanged*;
return C, $\{\mu_j\}$

Algorithm 1: k-Means (Lloyd's)

Key Properties

- **Requires k upfront** — use elbow method or silhouette
- **Spherical assumption** — sensitive to elongated shapes
- **Sensitive to init** — k-means++ improves this
- **Sensitive to outliers** — consider k-Medoids instead
- **Time:** $O(nkdt)$ — scales well to large data

k-Means — Algorithm & Properties

Input: Data $X = \{x_1, \ldots, x_n\}$, clusters k
Output: Cluster assignments C, centroids μ
Initialise $\mu_1, \ldots, \mu_k$ (random or k-means++);
repeat
    for *each point* x_i **do**
        $C(i) \leftarrow \arg\min_j \|x_i - \mu_j\|^2$;
    end
    for *each cluster* j **do**
        $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$;
    end
until *centroids unchanged*;
return C, $\{\mu_j\}$

Algorithm 2: k-Means (Lloyd's)

Elbow Method

Plot WCSS vs. k; choose the "elbow" point where adding more clusters gives diminishing returns:

$$k^* = \arg\ \text{elbow}\big(J(k)\big)$$

Table of Contents

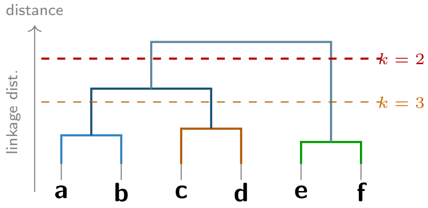
Hierarchical Clustering

Concept

Builds a **dendrogram** (tree of merges/splits). No need to pre-specify k — cut the tree at any level.

- **Agglomerative** (bottom-up): start with n clusters; merge closest pairs
- **Divisive** (top-down): start with 1 cluster; recursively split

Dendrogram Example:



Example: Gene Expression

Ward linkage groups co-regulated genes into *functional modules*. Dendrogram reveals pathway hierarchies without prior biological knowledge.

Hierarchical Clustering

Four Linkage Criteria:

Linkage	Formula
Single	$d(A,B) = \min_{a,b} \|a-b\|$
Complete	$d(A,B) = \max_{a,b} \|a-b\|$
Average	$d(A,B) = \frac{1}{\|A\|\|B\|} \sum_{a,b} \|a-b\|$
Ward's	$\Delta J = \frac{n_A n_B}{n_A+n_B} \|\mu_A - \mu_B\|^2$

Complexity: $O(n^2)$ space, $O(n^2 \log n)$ time. Use BIRCH for large datasets.

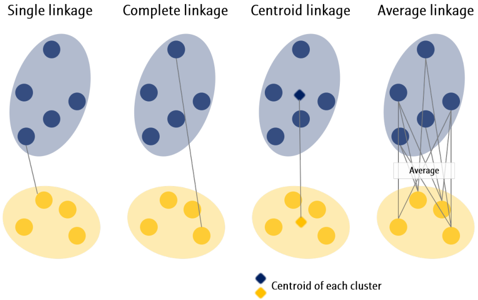


Table of Contents

Core Definitions

ε-neighbourhood:

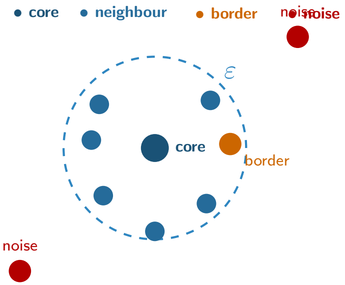
$$N_\varepsilon(p) = \{q \in D : \mathrm{dist}(p,q) \le \varepsilon\}$$

Core point: $|N_\varepsilon(p)| \ge \text{MinPts}$

Density-reachable: $\exists$ chain $p_1{=}p, \ldots, p_n{=}q$ where each p_i is core and $p_{i+1} \in N_\varepsilon(p_i)$

Density-connected: $\exists\, o$ from which both p and q are density-reachable

Point Classification Diagram:



DBSCAN — Density-Based Spatial Clustering

Parameters: ε (radius), MinPts (min neighbours).
Complexity: $O(n \log n)$ with spatial index.
No need to specify k.

Example: Geospatial Hotspots

Identify crime/event clusters of *irregular shape*. Isolated incidents are labelled as **noise** — no prior k.

Arbitrary-Shape Detection:

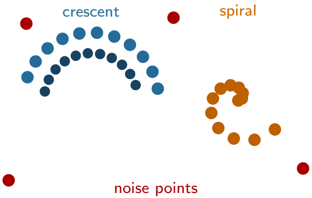


Table of Contents

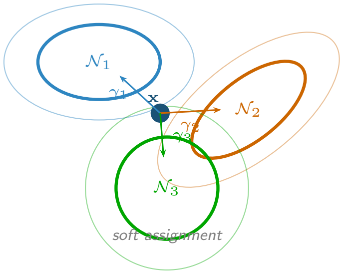
Gaussian Mixture Models (GMM)

Mixture Model Likelihood

$$p(\mathbf{x}) = \sum_{i=1}^{k} \pi_i \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_i, \, \boldsymbol{\Sigma}_i)$$

π_i: mixing weights, $\sum \pi_i = 1$
$\boldsymbol{\Sigma}_i$: covariance matrix (shape & orientation)

GMM vs. k-Means — Key Difference:



Gaussian Mixture Models (GMM)

EM Algorithm

E-step — compute responsibilities:

$$\gamma_{ij} = \frac{\pi_i \mathcal{N}(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{\ell=1}^{k} \pi_\ell \mathcal{N}(\mathbf{x}_j | \boldsymbol{\mu}_\ell, \boldsymbol{\Sigma}_\ell)}$$

M-step — update parameters:

$$\boldsymbol{\mu}_i^{\text{new}} = \frac{\sum_j \gamma_{ij} \mathbf{x}_j}{\sum_j \gamma_{ij}}, \qquad \pi_i^{\text{new}} = \frac{\sum_j \gamma_{ij}}{n}$$

Maximise log-likelihood $\mathcal{L} = \sum_j \log p(\mathbf{x}_j)$.

GMM vs. k-Means — Key Difference:

Properties

- **Soft** (probabilistic) cluster membership
- Handles **elliptical**, overlapping clusters
- Model selection via **BIC / AIC**
- EM guaranteed to increase $\mathcal{L}$ each step

Example: Fraud Detection

Flag transactions falling in low-probability regions of any GMM component as anomalies.

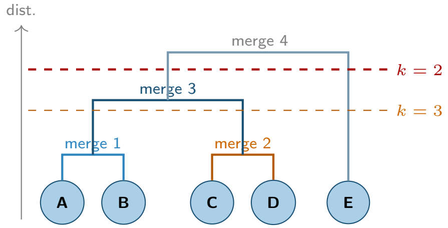
Table of Contents

AGNES — Agglomerative Nesting

Algorithm Idea

AGNES starts with each point as its own cluster and repeatedly merges the two *closest* clusters until only one cluster remains (or a stopping criterion is met).

Step-by-Step Merging:



AGNES — Agglomerative Nesting

Input: Points X, linkage $d(\cdot,\cdot)$
Assign each x_i to its own cluster C_i;
Compute all pairwise distances;
while *number of clusters* > 1 **do**
    $(A, B) \leftarrow \arg\min_{i \neq j} d(C_i, C_j)$;
    Merge A and B into new cluster;
    Update distance matrix;
end
return *Dendrogram*

Algorithm 3: AGNES (Agglomerative)

When to Use AGNES

- Need a **dendrogram** / tree structure
- Number of clusters **unknown** in advance
- Dataset is **small to medium** $(n < 10^4)$
- Biological, text, or document hierarchies

Ward's Merge Cost

$$\Delta J(A, B) = \frac{n_A\, n_B}{n_A + n_B} \left\| \boldsymbol{\mu}_A - \boldsymbol{\mu}_B \right\|^2$$

Merge the pair with the *smallest* ΔJ.

Table of Contents

Mean Shift Clustering

Kernel Density Estimate

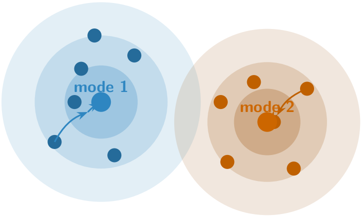
$$\hat{f}(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^{n} K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$$

Mean Shift vector (gradient step):

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_i K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right) \mathbf{x}_i}{\sum_i K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)} - \mathbf{x}$$

h: bandwidth (kernel radius), K: Gaussian or Epanechnikov kernel.

Gradient Ascent to Mode:



points shift toward density peaks

Properties

- **No need to specify** k modes auto-detected
- Finds **arbitrary shapes**
- Sensitive to **bandwidth** h
- **Complexity:** $O(n^2)$ per iteration — slow on large data

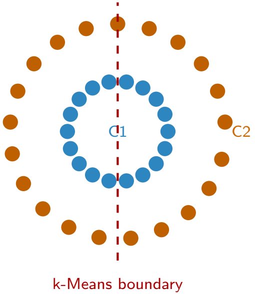
Table of Contents

Spectral Clustering

Idea

Use the **graph Laplacian** of the similarity (affinity) graph. Project data onto the leading eigenvectors, then run k-Means in the *spectral embedding*. Captures **non-convex manifold** structure.

Concentric Rings — Why k-Means Fails:



Spectral handles this correctly

Spectral Clustering

Graph Laplacian

Build affinity matrix W (e.g. Gaussian kernel):

$$W_{ij} = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

Degree matrix $D_{ii} = \sum_j W_{ij}$. Normalised Laplacian:

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$$

Embed: take k smallest eigenvectors of L_{sym}, stack as rows, normalise, then apply k-Means.

Input: Data X, clusters k, scaling σ
Output: Cluster labels for X

1. Construct Affinity Matrix W (e.g., Gaussian Kernel);
2. Compute Normalized Laplacian: $L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$;
3. Extract k smallest eigenvectors $u_1, \ldots, u_k$ to form matrix $U \in \mathbb{R}^{n \times k}$;
4. Normalize rows of U to unit length;
5. Cluster the rows of U using **k-Means**;

return *cluster labels*

Example: Social Networks

Nodes = users, W_{ij} = friendship strength. Spectral clustering finds **communities** with complex internal connectivity.

Table of Contents

Algorithm Comparison

Algorithm	Cluster Shape	Specify k?	Noise?	Complexity	Best Use Case
k-Means	Spherical	Yes	No	$O(nkdt)$	Large, convex, well-separated
AGNES	Arbitrary	Post-hoc	Partial	$O(n^2 \log n)$	Small data, need dendrogram
DBSCAN	Arbitrary	**No**	**Yes**	$O(n \log n)$	Irregular shapes, noisy data
GMM / EM	Elliptical	Yes	Partial	$O(nkdt)$	Overlapping, probabilistic
Mean Shift	Arbitrary	**No**	**Yes**	$O(n^2)$	Image segmentation, vision
Spectral	Non-convex	Yes	Partial	$O(n^3)$	Graph/manifold, ring shapes

Choosing the Right Algorithm

Validation Metrics:

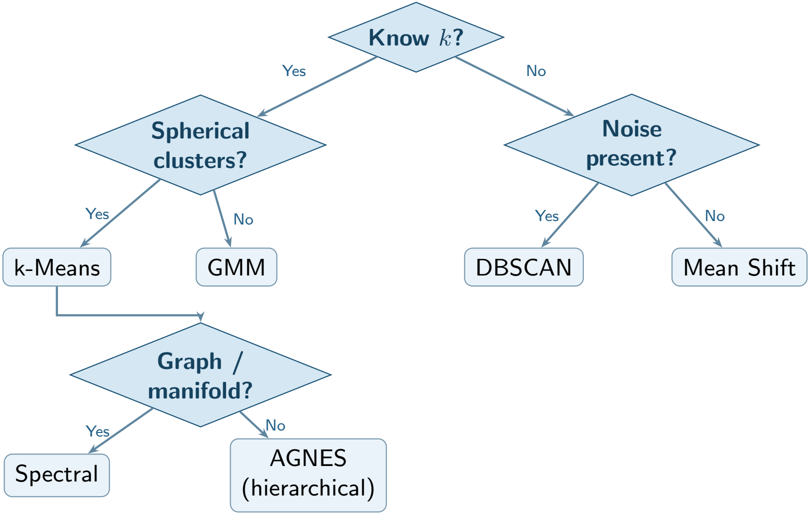
- Silhouette Score $s = \frac{b-a}{\max(a,b)} \in [-1, 1]$
- Davies-Bouldin Index
- Calinski-Harabász Index.

Golden Rule

No single algorithm wins.
Always:

- preprocess & normalise
- run 2–3 algorithms
- validate
- interpret in context

Choosing the Right Algorithm



Summary

Algorithms Covered

- **k-Means** — minimise WCSS; fast; spherical clusters
- **Hierarchical (AGNES)** — dendrogram; no k needed; $O(n^2)$
- **DBSCAN** — density-based; arbitrary shapes; handles noise
- **GMM / EM** — probabilistic; elliptical; soft membership
- **Mean Shift** — mode-seeking; no k; kernel density
- **Spectral** — graph Laplacian; manifold/ring shapes

Key Mathematical Objects

Algorithm	Objective
k-Means	$\min \sum \|x - \mu_i\|^2$
Ward's	$\min \Delta J = \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2$
DBSCAN	Density connectivity
GMM	$\max \sum \log \sum_i \pi_i \mathcal{N}$
Mean Shift	$\nabla \hat{f}(\mathbf{x}) \to$ mode
Spectral	Eigen-decomp of L

Rule of thumb: preprocess $\to$ try multiple algorithms $\to$ validate with Silhouette / DB index $\to$ interpret results in domain context.